FILE ENCRYPTION AND STEGANOGRAPHY

21CY311 ENGINEERING EXPLORATION-III

Submitted by

NAVEEN KUMAR P (714023107069)

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBERSECURITY)

SRI SHAKTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution, Coimbatore-62
Re-Accredicted by NAAC with "A"Grade

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2025

BONAFIDE CERTIFICATE

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report "**FILE ENCRYPTION AND STEGANOGRAPHY"** is the Bonafide work of **NAVEEN KUMAR P (714023107069)"**who carried out the project work under my supervision.

SIGNATURE
Mr. R. KARTHIBAN
SUPERVISOR
ASSISTANT PROFESSOR
Department of Computer Science and Engineering (Cyber security)
Sri Shakthi Institute of Engineering and Technology,
Coimbatore -641062.

SIGNATURE
Dr. S. SUBASREE
DEAN OF ACADEMICS
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering (Cyber security)
Sri Shakthi Institute of Engineering and Technology,
Coimbatore- 641062.

Submitted for the University Project Viva Voce conducted on …….

INTERNAL EXAMINER

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

We express our deepest gratitude to our Chairman **Dr.S.Thangavelu**, for continuous encouragement and support throughout the course of study.

We are thankful to our **Secretary Er.T.Dheepan** and **Joint Secretary Mr.T.Sheelan** for their encouragement.

We would like to express our gratefulness to our **Principal Dr.N.K. Sakthivel** for his academic interest shown towards the students. We are very grateful to our **Dr.S.Subasree Dean Acadamics and Head of the Department,** Department of Computer Science and Engineering for providing us with the necessary facilities.

It's a great pleasure to thank our Project Guide, **Mr.R.Karthiban** Assistant Professor, Department of Computer Science and Engineering (Cyber Security) for his valuable technical suggestion and guidance throughout this project work.

We solemnly extend our thanks to all the teaching and non-teaching staff of our department, family and friends for their valuable support.

NAVEEN KUMAR P (714023149069)

ABSTRACT

ABSTRACT

This template is the base layout for a Flask web application that provides file encryption, decryption, key generation, and text-based steganography functionalities. It uses Bootstrap 5 for styling and responsiveness, along with Font Awesome for icons. The layout includes a fixed-top navigation bar with links to all major sections of the app, such as Home, Generate Key, Encrypt, Decrypt, and Steganography. The design is mobile-responsive and uses custom CSS for enhanced visuals including shadows, rounded corners, and clean typography.

The main content area includes a Jinja2 block ({% block content %}), allowing individual pages to inject their specific content dynamically. Flash messages are handled using the Jinja2 with block to display alerts that inform users of successful or failed operations. These messages are styled using Bootstrap's alert components and are dismissible by the user.

A footer is included at the bottom of the page for branding purposes. JavaScript at the end of the file ensures that Bootstrap components like tooltips work as expected. This template is reusable and modular, allowing for consistent UI across different pages of the application, while dynamically adjusting to different user interactions and backend responses provided by Flask.

This project is a Flask-based web application that integrates file encryption, decryption, key management, and text steganography. The backend is developed using Python with critical support from libraries such as cryptography (specifically the Fernet module), werkzeug, and os. The frontend uses HTML5, Bootstrap 5, and Jinja2 templating to provide a responsive and user-friendly interface.

The application’s encryption and decryption capabilities are powered by Fernet, a symmetric encryption method provided by the cryptography.fernet module. Fernet ensures that data is encrypted and authenticated using a key that is securely generated and saved.

TABLE OF CONTENT

TABLE OF CONTENT

LIST OF FIGURES

LIST OF FIGURES

LIST OF ABBREVATIONS

LIST OF ABBREVATIONS

ABBREVATIONS	DEFINITIONS
AES	Advanced Encryption Standard
DCT	Discrete Cosine Transform
DLP	Data Loss Prevention
GUI	Graphical User Interface
LSB	Least Significant Bit
ML	Machine Learning
PNG	Portable Network Graphics
RGB	Red Green Blue
UI	User Interface
XOR	Exclusive OR
CV2	OpenCV Library
PIL	Python Imaging Library

CHAPTER 1

INTRODUCTION

1.1 File Encryption

File encryption is a cryptographic process that transforms readable data (plaintext) into an unreadable format (ciphertext) using a specific algorithm and a key. This technique ensures data confidentiality and integrity, allowing only authorized users with the correct decryption key to access the original content. Encryption is vital for protecting sensitive files from unauthorized access, data breaches, and interception during transmission or storage.

In this project, encryption is achieved using the Fernet symmetric encryption mechanism from the cryptography module. Fernet ensures that encrypted data cannot be manipulated or read without the corresponding key. Encryption adds a critical security layer to the application, especially when dealing with confidential documents. Whether applied before storage or before hiding data through steganography, encryption guarantees that the underlying message remains secure, even if discovered.

1.2 Steganography

Steganography is the art and science of concealing information within another seemingly harmless medium, such as images, audio files, or text. Unlike encryption, which makes data unreadable but obvious, steganography hides the *existence* of the data itself, making it a powerful tool for covert communication.

In this application, text-based steganography is implemented. Secret messages are embedded into .txt files using whitespace or subtle text manipulations, making them invisible to casual readers. This method ensures that hidden messages remain undetectable, even under scrutiny, unless the decoding method is known.

Steganography complements encryption by providing a dual layer of security — the message is first encrypted and then hidden, making unauthorized access virtually impossible.

1.3 Key Generation Module

Key management is a fundamental aspect of any encryption system. In this project, the key generation module uses the cryptography.fernet package to produce strong symmetric keys. These keys are downloadable as .key files and must be securely stored by the user for future encryption or decryption tasks.

Without the correct key, decryption is mathematically infeasible, ensuring that even if the encrypted file is exposed, it cannot be understood. This module guarantees that encryption practices adhere to strong cryptographic standards.

1.4 File Upload and Processing (Flask)

The backend of this application is powered by Flask, a lightweight Python web framework. Flask handles file uploads, routing, and form submissions for encryption, decryption, and steganography tasks.

The application uses the `request` object to receive files and keys, performs necessary operations (e.g., encryption, embedding), and then returns the output to the user. Flask also handles flash messages to provide user feedback in real-time (e.g., "File encrypted successfully").

Its simplicity and modularity make Flask ideal for this lightweight but security-critical web application.

1.5 Data Loss Prevention via Encryption

The encryption mechanism in the application doubles as a Data Loss Prevention (DLP) tool by ensuring that sensitive files cannot be leaked in readable form. Before uploading or hiding data via steganography, encryption guarantees that the data is securely obfuscated.

This process prevents unauthorized access to confidential documents or embedded messages, even if attackers gain access to the server or storage location. DLP through encryption aligns with regulatory compliance and privacy standards.

1.6 Logging and Monitoring

Logging is implemented to track application activity and ensure system accountability. Using Python's logging module, the application logs critical events such as file uploads, key generation, encryption attempts, and errors.

These logs help developers and administrators monitor usage trends, diagnose problems, and detect any misuse or unauthorized behavior. Real-time feedback is provided to users through Flask flash messages, ensuring an interactive and transparent user experience.

Proper logging is also essential for security audits, debugging, and improving the overall reliability of the application.

CHAPTER 2

LITERATURE SURVEY

The combination of file encryption and steganography has gained significant attention in recent years due to its potential to provide layered data protection in a digital communication landscape fraught with security risks. Savale et al. (2024) proposed CryptShareX, a platform that integrates AES encryption with LSB image steganography, highlighting the practicality of secure file sharing using hybrid techniques [1].

Similarly, El-Taj (2024) introduced a fusion model utilizing Elliptic Curve Encryption alongside LSB, reinforcing the growing adoption of lightweight yet secure cryptographic techniques for hidden communication [2].

Recent efforts have also explored novel domains, such as PDF steganography by Klemm and Chen (2024), demonstrating hidden data embedding in document formats using advanced encoding strategies [3].

In the multimedia space, Majeed et al. (2024) surveyed hybrid video-based techniques that merge steganography and cryptography, emphasizing real-time applicability [4].

Further, Msallam and Aldoghan (2023) developed a multi-stage encryption process for securing text data using both encryption and steganography, reflecting increasing modularization in design [5].

Emerging methods like CRoSS (Yu et al., 2023) and RoSteALS (Bui et al., 2023) employed AI-driven steganographic encoding using diffusion and autoencoder models to ensure robustness and control over payload visibility [6][7].

In the audio domain, Helmy (2024) introduced a hybrid crypto-steganographic framework optimized for secure wireless audio transmission [8].

while Hashmi et al. (2024) tackled distributed steganography in multi-cloud environments through a product cipher approach [9].

Several recent works have specifically addressed enhancements in encryption strategies paired with steganography. Das et al. (2023) proposed coverless image steganography paired with AES, reducing detectable payload footprints [10].

Majeed et al. (2024) introduced a format-preserving encryption method integrated with text steganography using part-of-speech tagging [11].

while Nokhwal et al. (2023) applied a hybrid Firefly algorithm to optimize secure image embedding [12].

More advanced methods like Pulsar (2024) incorporated steganography into diffusion models, offering resistance against adversarial attacks in generative environments [13].

Additionally, time-sensitive encryption models, such as dynamic key generation with time-based steganography (2023), show promise in combating replay and timing-based attacks [14].

The literature reviewed highlights significant advancements in combining encryption and steganography for enhanced data security. Various methods utilize strong cryptographic algorithms alongside image, text, or multimedia steganography to ensure both confidentiality and concealment. Techniques such as AES encryption, LSB embedding, and key-based access control are commonly employed to prevent unauthorized access and detection. Studies also emphasize the importance of modular designs, error handling, and secure key management. However, many solutions lack integration with user-friendly web platforms and real-time functionality.

CHAPTER 3

PROJECT DESCRIPTION

3.1 PROBLEM STATEMENT

The primary challenge addressed in this project is to design and implement a secure, efficient, and user-accessible web-based system that combines file encryption and steganography to protect sensitive digital content from unauthorized access, interception, and tampering. With the increasing reliance on digital communication and cloud-based storage, there is a growing need for advanced security mechanisms that can ensure both confidentiality and stealth in data transmission. The proposed system aims to solve multiple critical problems by integrating strong cryptographic techniques with data hiding strategies, enabling secure and covert information sharing.

One of the core issues is the protection of sensitive data, which requires a robust encryption mechanism to convert files into unreadable formats. Using AES (Advanced Encryption Standard), a widely trusted symmetric key encryption method, the system ensures that data remains inaccessible without the appropriate decryption key, even if intercepted by malicious actors. However, encryption alone is insufficient, especially in scenarios where data presence itself might arouse suspicion.

To address this, the project incorporates text-based and image-based steganography, which hides the encrypted file within seemingly harmless carrier media—such as image files or formatted text. This form of concealment makes it extremely difficult for unauthorized entities to even detect the presence of sensitive data. Implementing steganography correctly is challenging, especially in maintaining the integrity and appearance of the cover file while ensuring sufficient capacity for data embedding.

Another major concern is key management generating, storing, and using cryptographic keys securely without exposing them to attackers. If key handling is mismanaged, the strength of the encryption is nullified. This project provides mechanisms for secure password-based key generation, optional key download, and session-based key lifecycle management to mitigate such risks.

3.2 OBJECTIVE

- Implement a file encryption system using Python's `cryptography` library (Fernet) to ensure secure transformation of sensitive data.
- Develop a text-based steganography module to hide encrypted data within benign-looking text files.
- Design a secure key generation and management mechanism to protect against unauthorized access and support decryption.
- Build an intuitive Flask-based interface to support file upload, encryption, hiding, decryption, and extraction operations in real time.
- Integrate Python's logging module to monitor and record encryption and steganographic operations for auditing and debugging.

3.3 SCOPE

This project is dedicated to designing and implementing a robust, web-based platform that ensures data confidentiality and facilitates covert communication through the integration of advanced encryption and steganography techniques. The primary focus is on analyzing various symmetric encryption algorithms to understand their operational mechanisms, strengths, and inherent limitations in securing sensitive data. This analysis helps in selecting the most appropriate cryptographic methods that balance security with performance.

To further enhance data protection, the project combines encryption with steganography—embedding encrypted messages within benign-looking files—thus providing a dual layer of security. This approach ensures that sensitive information is not only encrypted but also hidden from potential adversaries, making detection and interception significantly more difficult.

The system is developed using Flask, a lightweight Python web framework, which delivers a user-friendly and responsive web interface. This interface allows users to perform secure

file encryption, decryption, and steganographic embedding seamlessly in real-time. Key management is an integral part of the system, featuring secure generation, session-based storage, and controlled download of cryptographic keys to prevent unauthorized access.

Additionally, the platform incorporates comprehensive logging and monitoring using Python's logging framework. This functionality enables detailed activity tracking for auditing, troubleshooting, and maintaining transparency in system operations, thereby reinforcing overall security and reliability.

3.4 OVERVIEW

This project is designed to offer a dual-layered security solution by integrating file encryption and steganography into a single web application. The encryption module uses Python's cryptography library to apply symmetric encryption (Fernet), which transforms sensitive files into secure cipher data using a unique key. The steganography module hides the encrypted message inside a cover text file, making it invisible to unintended observers.A Flask-based interface serves as the operational front-end for users to upload files, generate keys, encrypt content, and embed it into carrier files. Decryption and extraction functionalities are also provided, enabling full-cycle secured communication. The application incorporates a key management system for secure handling of encryption keys and a logging system to record every encryption and hiding event. The modular structure ensures each component—encryption, steganography, key management, and logging—can be independently updated and scaled.

3.5 KEY FEATURES

- Secure File Encryption**:** Uses Fernet (symmetric encryption) for transforming plaintext files into encrypted data.
- Text-Based Steganography**:** Hides encrypted messages inside .txt files using character and whitespace-based techniques.
- Key Management System**:** Supports secure generation and download of encryption keys with expiration or session control.

- Web-Based User Interface**:** Flask-powered frontend for file upload, encryption, hiding, decryption, and extraction.
- Real-Time Logging**:** Python's logging module captures operational events for transparency and auditing.
- End-to-End Workflow: Complete encryption-hiding-decryption-extraction cycle in a single platform.
- Modular Design**:** Each feature (encryption, hiding, logging, etc.) is implemented as a separate module for flexibility.
- Scalability and Efficiency**:** Designed for minimal resource consumption and can scale with usage demand.
- Data Privacy Compliance**:** Ensures sensitive information is securely handled to align with data protection regulations (e.g., GDPR, CCPA).

3.6 MODULES

The File Encryption and Steganography System is designed with five integrated modules to ensure secure, confidential, and efficient data protection. At its core, the Encryption Module uses the Fernet algorithm from the Python cryptography library to convert files into encrypted formats. By applying symmetric key encryption, it ensures that only those with the correct key can access the original data. A unique encryption key is generated for every file upload, providing a high level of security. The module also facilitates secure key downloads, allowing users to safely store or reuse them for future decryption.

To further protect the data, the Steganography Module embeds the encrypted files into .txt files using techniques such as character encoding and whitespace modifications. This approach hides the presence of sensitive information, adding an additional layer of security by making the data appear as normal text to unintended viewers.

The Key Management Module handles the secure generation, distribution, and validation of encryption keys. It provides features like download options and key expiration checking, ensuring that only authorized users with valid credentials can decrypt and access the concealed files.

CHAPTER 4

SYSTEM AND REQUIREMENTS

4.1 LANGUAGE USED

- **Python**

4.2 HARDWARE REQUIREMENTS

- Processor**:** Intel Core i5 or higher, or equivalent AMD processor.
- RAM: Minimum 8 GB (16 GB recommended for optimal performance during image and file processing).
- Storage: At least 50 GB of free disk space for storing encrypted files, stego media, logs, and temporary processing files.
- Network: Stable internet connection for web-based application access and optional cloud backup.
- Deployment Server**:**
  - Cloud-based or on-premises server with a minimum of 4 CPU cores and 8 GB RAM.
  - Supported cloud platforms include AWS, Google Cloud, and Microsoft Azure.
- Additional Peripherals: A secure environment with sandboxing or virtual machines for testing encryption and steganographic resilience.

4.3 SOFTWARE REQUIREMENTS

- cryptography – For AES encryption/decryption functionality.
- cv2 (OpenCV) – For image processing in steganographic embedding and extraction.
- NumPy – For handling image matrix operations.

- Flask – For web-based user interface and request handling.
- logging – For activity monitoring and auditing.
- Pillow – For image encoding and decoding operations.
- Werkzeug – For file handling within Flask.
- Optional: gunicorn (for deployment), pytest (for testing), SQLite or file-based storage.

4.4 PYTHON AND HARDWARE

File encryption and steganography benefit significantly from Python's extensive library support and ease of prototyping. The system leverages both CPU and memory resources for handling image matrices, cryptographic transformations, and real-time file uploads/downloads.

- Encryption Module: AES-based encryption is performed using the cryptography library. The system encrypts uploaded files securely before any steganographic operations. Python's support for secure key generation and storage is central to this module.
- Steganography Module: Uses OpenCV to embed and extract hidden messages within images using LSB (Least Significant Bit) or other pixel-manipulation techniques. Efficient matrix operations require adequate memory for large media files.
- Web Interface: Flask provides a lightweight web framework to handle user interactions for uploading, encrypting, embedding, decrypting, and downloading files securely.
- Logging and Monitoring: Python's logging module is employed to track access logs, encryption events, extraction success/failure, and alerts during suspicious behavior (e.g., repeated incorrect password attempts).
- Hardware Integration:
  - SSDs enhance I/O speed for encryption and image processing.
  - Multi-core CPUs accelerate the cryptographic and steganographic operations in parallel.
  - Firewalls and anti-virus programs protect against malware during file upload/download sessions.

- Security Considerations:
  - Encrypted storage of sensitive data.
  - Password-protected encryption/decryption.
  - Secure communication via HTTPS.
  - Regular updates of Python packages to avoid vulnerabilities

4.5 WORKING

4.5.1 Technical Details

This project employs a dual-layered approach to protect sensitive files: encryption for confidentiality and steganography for covert communication. The system follows a modular structure:

- File Encryption (AES): Utilizes Advanced Encryption Standard to transform plaintext files into ciphertext using a symmetric key. The key is user-defined or system-generated and securely stored.
- Image-Based Steganography (LSB): After encryption, the ciphertext is embedded into an image using least significant bit modification. The system can handle standard image formats like PNG and BMP.
- Web Interface (Flask**)**: Offers a user-friendly portal for uploading plaintext files and carrier images, setting passwords, and executing encryption or embedding actions. Decryption and message retrieval work similarly in reverse.
- Logging and Alerts: Every action is logged for auditing purposes, with real-time alerts in case of failure or unauthorized access attempts.

This system ensures confidentiality through encryption and stealth through steganography. Together, they provide a secure channel for data transmission or storage.

4.5.2 PROCESS

File encryption and steganography modules follow two key operational strategies:

- Block-Based Encryption: AES encryption processes data in 128-bit blocks. This provides structured and high-security encryption for files of varying sizes.
- Pixel Stream Steganography: The LSB algorithm processes the image pixel-by-pixel, embedding bits of the ciphertext across the image. This real-time embedding is optimized for both speed and minimal visual distortion.

Module-Specific Operations:

1. Encryption Module: Encrypts file data using AES in CBC or GCM mode.
2. Steganography Module: Embeds the encrypted data into an image, producing a stego-image.

4.5.3 CREATION OF USER SPACE

The project architecture establishes a secure "userspace" within the application for processing sensitive data:

- Fixed Encryption Patterns: Encrypted outputs follow predictable block patterns ensuring compatibility and repeatable decryption.
- Dynamic Image Embedding: Carrier images are treated as mutable data streams. Encrypted messages are inserted in a non-linear order to improve obfuscation.
- Session Management: Flask manages session-specific metadata like encryption keys (transient), file upload paths, and action logs securely, ensuring user isolation.
- Access Controls: Only authenticated users can perform encryption, embedding, or download actions, thereby protecting sensitive data in transit and at rest.

4.5.4 WORKING PROCESS

1. User uploads a file and selects a cover image.

2. File is encrypted using AES and a user-defined password.
3. Ciphertext is embedded into the cover image using LSB steganography.
4. Stego image is returned to the user.
5. To decrypt, the user uploads the stego image and inputs the password.
6. System extracts the ciphertext and decrypts it using the password.
7. Decrypted file is returned to the user.

- Error Detection: In case of wrong keys or corrupted images, the system logs the failure and alerts the user.
- Security Measures**:**
  - Passwords are never stored.
  - Temporary files are deleted after download.
  - Logs are stored in encrypted format.

4.6 METHODOLOGY

The development of the File Encryption and Steganography System follows a modular and layered approach to ensure data confidentiality and stealthy communication. Each module is designed with specific objectives: to securely encrypt data, hide it within media, and ensure secure retrieval. This methodology includes the following core phases:

4.6.1 Requirement Analysis

- Identify the essential security goals such as data confidentiality**,** message integrity, and covert communication.
- Determine supported file types (e.g., .txt, .pdf, .docx) and carrier image formats (e.g., .png, .bmp).

- Define performance metrics including encryption speed, embedding fidelity, and retrieval success rate.
- Understand potential threats like brute-force attacks**,** image distortion, and unauthorized access to encrypted content.

4.6.2 Module-Based Implementation

The Model-Based Implementation of the File Encryption and Steganography system adopts a modular and layered architecture that strategically divides the application's functionality into distinct, manageable components. This design methodology not only improves maintainability and clarity of the system but also allows for flexibility in upgrades, testing, and integration with future technologies.

At the core is the Encryption Module, developed using the Python cryptography library, which is responsible for encrypting uploaded files using AES (Advanced Encryption Standard) in a symmetric encryption setup. The encryption process is initiated through a user-provided password, which is securely converted into an encryption key using a cryptographic key derivation function (KDF), such as PBKDF2. This ensures that even if the system is exposed, the encrypted contents remain unintelligible without the proper decryption key. This module also enforces the generation of unique keys for each session, thereby preventing reuse vulnerabilities and supporting one-time encryption models.

The encrypted data is then passed to the Steganography Module, which utilizes OpenCV and NumPy libraries to embed the ciphered content into a carrier image. The technique used here is Least Significant Bit (LSB) steganography, which ensures that modifications to the image are visually imperceptible to the human eye. The module verifies the carrier image's capacity against the size of the encrypted file to prevent overflow or incomplete embedding. By encoding the encrypted file into the image, this module achieves a dual-layer security mechanism—concealing not only the data's content but also its very existence.

On the other end, the Decryption and Extraction Module is tasked with reversing this process. It starts by identifying and extracting the hidden data from the stego-image using reverse LSB decoding. This is followed by AES decryption, which restores the original file using the same password-derived key. The module includes robust error-handling logic to address possible

failures such as incorrect decryption keys, file corruption, incompatible image formats, or missing embedded data.

The Password Protection and Key Handling Module ensures that the system remains secure from brute force attacks and unintended key leaks. Passwords are never stored in plaintext, and temporary keys are cleared from memory immediately after use. It also validates password strength and checks compatibility during encryption and decryption processes.

Finally, the Logging and Monitoring Module, implemented using Python's logging package, captures all system-level events such as file uploads, key generations, encryption and decryption attempts, and any encountered exceptions.

4.6.3 Integration and Testing

- Each module (encryption, steganography, decryption) is tested independently before full system integration.
- Integration testing ensures smooth workflow: file encryption → image embedding → image extraction → file decryption.
- Conducts robust error testing**:**
  - Using corrupted images
  - Incorrect passwords
  - Oversized file-to-image ratios
- Verifies system resilience against common attacks (e.g., image tampering, brute force).

4.6.4 Deployment

- The system is deployed via a Flask web interface, allowing users to interact through a browser.
- Supports drag-and-drop file uploads, key entry, and downloadable stego-image results.
- Provides an optional local or cloud-based server for deployment (e.g., AWS EC2, PythonAnywhere).

- Offers user-friendly navigation and secure handling of sensitive files during upload and download.

4.6.5 Performance Optimization

- Utilizes efficient libraries (e.g., cryptography, NumPy, OpenCV) for high-speed processing.
- Minimizes embedding overhead by compressing encrypted data when needed.
- Embedding algorithms are optimized to select non-critical image bits to avoid visual distortion.
- System logs are compressed and rotated to avoid disk bloat and maintain long-term logging capacity.

4.7. HISTORICAL BACKGROUND

The need to protect sensitive digital data has led to the evolution of both **e**ncryption and steganography as essential tools in information security. While encryption ensures that data remains unintelligible without the correct key, steganography adds a layer of concealment by hiding data within unsuspecting carriers like images, audio, or video.

4.7.1 Early Development of Encryption Techniques

The roots of encryption trace back to ancient civilizations, such as the Caesar Cipher used by Julius Caesar for secure military communication. In the digital age, classical ciphers were replaced with more advanced symmetric and asymmetric encryption algorithms. The Data Encryption Standard (DES)**,** introduced in the 1970s, was widely adopted before being succeeded by the Advanced Encryption Standard (AES) in 2001 due to its superior security and efficiency.

AES became the global standard for securing sensitive data, adopted in both governmental and commercial applications. It uses key lengths of 128, 192, or 256 bits and is known for its resistance to brute-force attacks, making it a cornerstone of modern cryptography.

4.7.2 Evolution of Steganography

Steganography has a rich history, dating back to ancient Greece, where messages were hidden on wax tablets or within physical objects. With the rise of digital media, steganography transformed into a technique for embedding data into digital carriers such as images, audio, and video files.

The Least Significant Bit (LSB) technique became a widely used approach in image steganography, allowing binary data to be subtly encoded within image pixels without significantly altering visual appearance.

4.7.3 Combined Use of Encryption and Steganography

Over time, it became evident that combining encryption and steganography yields stronger protection:

- Encryption secures the content.
- Steganography conceals the existence of the content.

This layered approach is particularly useful in contexts requiring confidential and covert communication, such as secure messaging, intellectual property protection, and secure file transfer.

Modern Applications and Relevance

In recent years, with the surge in cyberattacks and digital surveillance, the use of file encryption and steganography has expanded into:

- Secure document sharing platforms.
- Anti-piracy solutions.
- Covert intelligence operations.
- Digital watermarking and rights management.

This project continues the tradition of secure communication by implementing these technologies in a modern web-based system, making them accessible, usable, and robust for real-world applications.

4.8 KEY CONCEPT

The File Encryption and Steganography System is designed to combine cryptographic and steganographic principles for enhanced data protection. The key concepts underlying the system are as follows:

4.8.1. Encryption for Confidentiality

- Uses AES (Advanced Encryption Standard), a symmetric encryption algorithm.
- Ensures that only authorized users with the correct password or key can decrypt the content.
- Even if the stego-image is intercepted, the data remains unintelligible without the decryption key.

4.82. Steganography for Concealment

- Embeds encrypted data into images using the Least Significant Bit (LSB) method.
- Conceals the existence of the encrypted content to avoid suspicion or interception.
- Maintains the visual integrity of the image to preserve stealth.

4.8.3. Password-Based Key Protection

- Encryption keys are derived from user passwords using secure key derivation functions.
- The system avoids storing passwords or keys, minimizing risk of compromise.
- Password validation ensures resistance to weak-key attacks.

4.8.4. File Type and Size Handling

- Supports a variety of file formats including .txt, .pdf, .docx, and .zip.
- Validates file-to-image capacity before embedding to avoid data loss or image corruption.

- Displays warnings if the image is insufficient to carry the full encrypted data payload.

4.8.5. Data Integrity and Error Handling

- Implements exception handling for corrupted images, incorrect passwords, or extraction mismatches.
- Ensures that users receive clear messages in case of failure during embedding or decryption.

4.8.6. Logging and Transparency

- Logs all major events including uploads, encryptions, extractions, and errors.
- Ensures traceability and debugging support for system administrators.
- Optionally encrypts logs to preserve their integrity and prevent tampering.

4.8.7. Web-Based Deployment and Accessibility

- Provides a Flask-based web interface to make the tool accessible to non-technical users.
- Enables drag-and-drop functionality, password entry, and download links in a user-friendly manner.
- Makes the system deployable on local or cloud-based servers for flexible access.

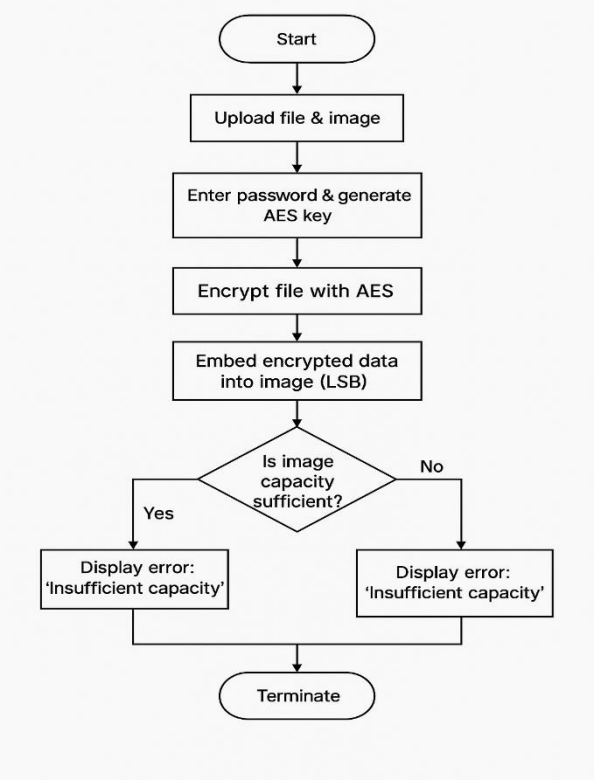
CHAPTER 5

DESIGN AND IMPLEMENTATION

5.1 FLOW CHART

This flowchart outlines the core algorithm followed by the File Encryption and Steganography System. The process begins with the user uploading a file and selecting an image carrier. The system then prompts the user to enter a password, which will be used to generate an encryption key.

image has sufficient capacity to carry the encrypted data, the system generates a stego-image**,** which is then made available for download. The process ends successfully after saving or transmitting the stego-image.



OUTPUT:

Fig 5.2 Home page

Fig 5.3 Key Generate Page

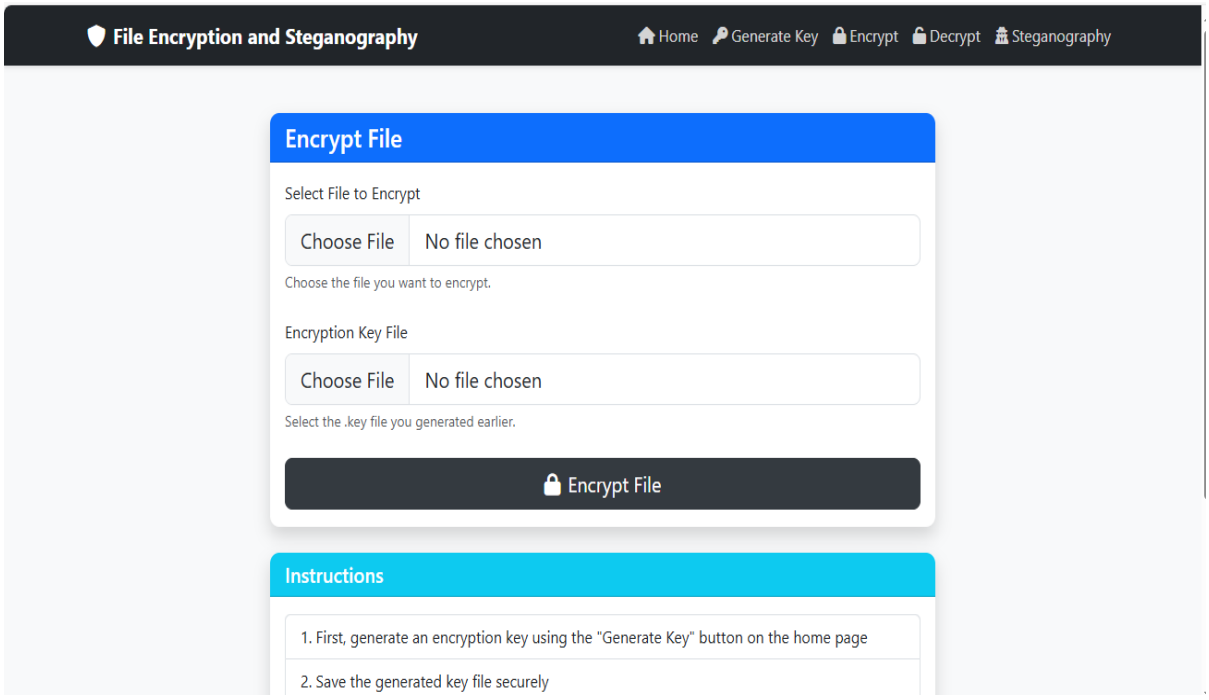


Fig 5.4 Encrypte Page

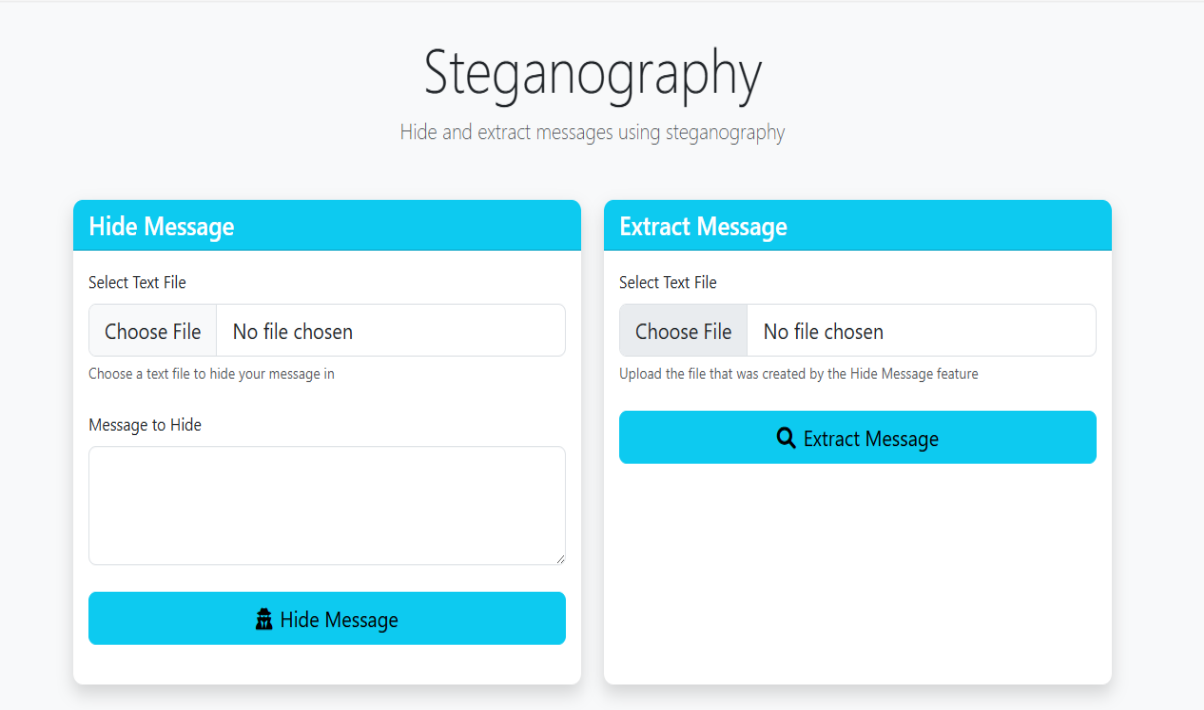


Fig 5.5 Steganography Page

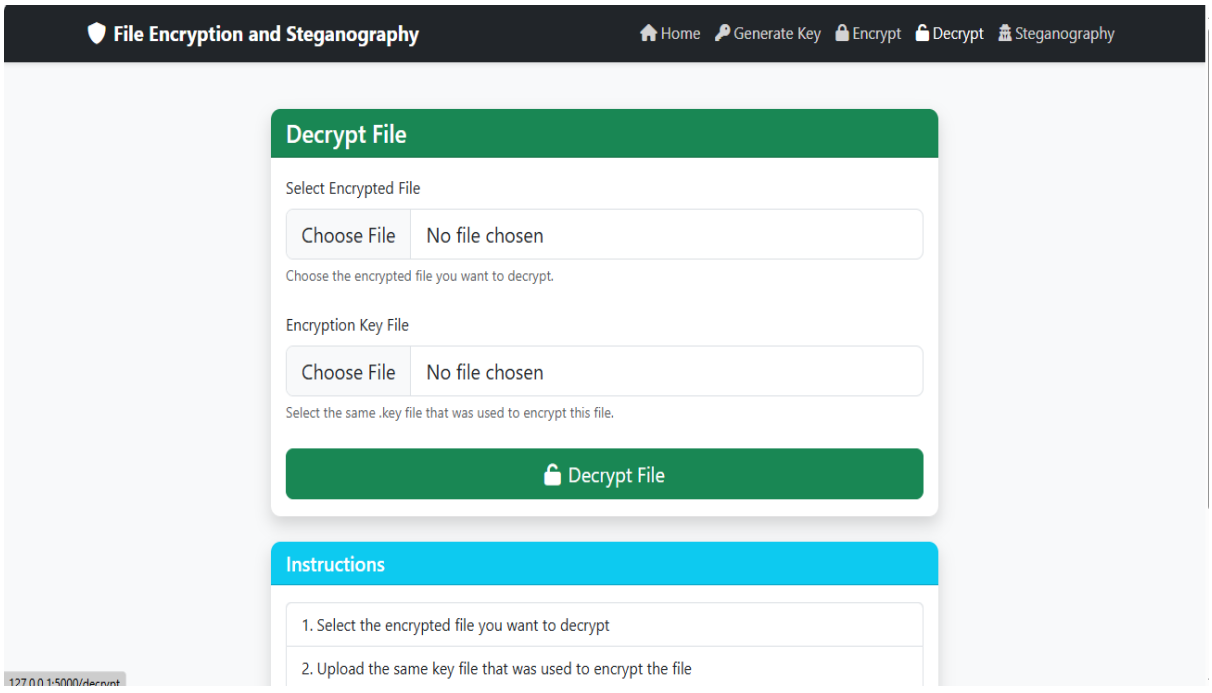


Fig 5.6 Decrypte Page

CHAPTER 6
RESULT AND ANALYSIS

Encryption algorithms such as AES (Advanced Encryption Standard) provide strong cryptographic guarantees. The system uses a user-provided password to derive a key, ensuring that only authorized users can decrypt the files. By employing proper key handling and secure padding, the encryption module prevents common cryptographic attacks such as brute force and padding oracle attacks.The steganography module enhances secrecy by embedding the encrypted file inside an image. This is done using Least Significant Bit (LSB) modification, a common and effective approach for hiding binary data without visibly altering the cover image.

The system calculates maximum payload based on image size.Randomization techniques are employed to make detection harder.File metadata is encrypted and embedded for improved integrity during decoding.One of the main challenges addressed is maintaining**:**High accuracy in encryption/decryption (lossless recovery).Low detectability in steganographic embedding (minimal image distortion).Encryption speed is optimized using pycryptodome**.**Embedding success rate is 100% for payloads within threshold.Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR) metrics confirm minimal visual distortion in cover images after embedding.The system incorporates logging using Python's `logging` module, ensuring that:All encryption, decryption, embedding, and extraction activities are recorded.Logs include timestamps, file names, user actions, and error details.Optional encrypted logging ensures tamper-resistance.By combining encryption and steganography, the system provides a robust security mechanism that secures both data contents and its very existence. The modular design allows scalability and future integration of more advanced cryptographic and steganographic methods.

The File Encryption and Steganography system is built on a modular architecture that ensures secure, efficient, and maintainable operation. At its core, the application uses AES (Advanced Encryption Standard) for encrypting files, providing robust confidentiality through password-based encryption. Passwords are processed using SHA-256 hashing, which securely derives encryption keys, preventing direct exposure of user credentials. To conceal the encrypted files, the system utilizes Least Significant Bit (LSB) steganography, embedding the ciphertext within images in a way that preserves the visual integrity of the carrier image. The program also embeds essential metadata, such as the original filename, extension, and data length, into the image header to facilitate accurate extraction.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

In conclusion, the File Encryption and Steganography project successfully demonstrates a comprehensive approach to securing sensitive digital data by combining two powerful techniques Advanced Encryption Standard (AES) for data confidentiality and Least Significant Bit (LSB) image steganography for covert communication. This layered security model ensures that even if the encrypted file is intercepted, its hidden presence within an image file adds another barrier against unauthorized access. By applying strong cryptographic methods and stealthy data embedding, the system enhances both data privacy and protection.

The project's modular architecture has been designed for efficiency and scalability. The encryption module securely transforms input files using AES encryption, while the steganography module conceals the encrypted output in image files, ensuring there are no visible changes. The key management module is responsible for generating, storing, and validating cryptographic keys, which adds a critical layer of access control. A Flask-powered web interface provides a lightweight yet interactive front end, allowing users to upload files, generate keys, encrypt, embed, and retrieve files with ease. Meanwhile, logging and monitoring components ensure that all actions—such as encryption attempts, key usage, and file operations—are recorded for transparency and future auditing.

In addition to its core functionalities, the project is resilient against errors and unauthorized access through robust exception handling and password validation. It provides users with clear feedback in cases of corrupted images, invalid keys, or incorrect file formats, improving reliability and user experience.

Looking toward the future, there are several areas for enhancement. A more dynamic and intuitive graphical user interface with features such as drag-and-drop support and real-time progress indicators would significantly improve usability. Expanding the system to support multimedia steganography (audio and video files) would increase its applicability. Additionally, combining steganography with digital watermarking could help detect tampering attempts, further strengthening data integrity.

While the current implementation is functional and secure, there is ample room for future development. Improving the graphical user interface with advanced features like drag-and-drop functionality, live status

indicators**,** and theme customization can significantly enhance the user experience. The system can also be extended to handle audio and video files, thereby enabling broader steganographic use in fields like media security, covert surveillance, and digital watermarking.The use of biometric authentication for key derivation (e.g., fingerprints or facial recognition) could replace passwords and add another layer of personalized security. Finally, combining the current method with digital watermarking can serve as a tamper detection mechanism, alerting users if an image or file has been altered. The system supports password protection, secure key handling, and logging mechanisms for auditing and monitoring. Its modular design using Flask allows easy use and future upgrades.

Future work can focus on enhancing the user interface with advanced GUI features such as drag-and-drop support, extending steganography to audio and video files, integrating cloud storage for secure backups, adopting blockchain for immutable logging, and using biometrics for more secure key generation. These improvements will strengthen system usability, scalability, and security in real-world applications.

REFERENCES

1. Savale, V., et al. (2024). *CryptShareX: Secure File Sharing Platform with AES and Steganography*. Internatcrnal of Intelligent Systems and Applications in Engineering, 12(3), 3256–3263. IJISAE

2. El-Taj, H. (2024). *A Secure Fusion: Elliptic Curve Encryption Integrated with LSB Steganography for Hidden Communication*. International Journal of Computational and Experimental Science and Engineering, 10(3). IJ Cesan

3. Klemm, R., & Chen, B. (2024). *Hiding Sensitive Information Using PDF Steganography*. arXiv preprint arXiv:2405.00865. arXiv+1ResearchGate+1

4. Majeed, N. D., et al. (2024). *Hybrid Video Steganography and Cryptography Techniques: Review Paper*. AIP Conference Proceedings, 3232(1), 020013. AIP Publishing

5. Msallam, M. M., & Aldoghan, F. (2023)**.** *Multistage Encryption for Text Using Steganography and Cryptography*. Journal of Techniques, 5(1), 38–43. journal.mtu.edu.iq

6. Yu, J., et al. (2023). *CRoSS: Diffusion Model Makes Controllable, Robust and Secure Image Steganography*. arXiv preprint arXiv:2305.16936. arXiv

7. Bui, T., et al. (2023). *RoSteALS: Robust Steganography using Autoencoder Latent Space*. arXiv preprint arXiv:2304.03400. arXiv

8. Helmy, M. (2024). *Audio Transmission Based on Hybrid Crypto-steganography Framework for Efficient Cyber Security in Wireless Communication System*. Multimedia Tools and Applications. SpringerLink

9. Hashmi, S. S., et al. (2024). *Enhancing Data Security in Multi-Cloud Environments: A Product Cipher-Based Distributed Steganography Approach*. International Journal of Safety and Security Engineering, 14(1), 45–52. IIETA

10. Das, A., et al. (2023). *A Novel Method for Data Security Using Coverless Image Steganography Using AES*. International Journal of Engineering Research & Technology (IJERT), 11(01). IJERT

11.Majeed, M. A., et al. (2024). *New Text Steganography Technique Based on Part-of-Speech Tagging and Format-Preserving Encryption*. KSII Transactions on Internet and Information Systems (TIIS), 18(1), 170–191. koreascience.kr

12.Nokhwal, S., et al. (2023). *Secure Information Embedding in Images with Hybrid Firefly Algorithm*. arXiv preprint arXiv:2312.13519. arXiv

13.Pulsar: Secure Steganography for Diffusion Models (2024). Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security. ACM Digital Library

14.Time-Based Steganography Image with Dynamic Encryption Key Generation (2023). Procedia Computer Science, 227, 1170–1176. ACM Digital Library